شرح سطر بسطر ٨ — شاشة 26: إنشاء الملف، الخطوة 2 (اجعل الملف لك)

مشروع رِزمة · Flutter · كل سطر مع سبب كتابته بهذه الطريقة

الإطار في `إعداد الملف 26` :Figma والمرجع `design_spec/screens_25-29_and_19.js` . الخطوة الأخيرة قبل البدء: اختيار **لون الشخصية** (مع معاينة فورية)، و**رمز سري** اختياري (مطفأ افتراضيًا)، و**رمز صف** اختياري.

عند «ابدأ التعلّم» يُنشأ الملف في القاعدة ويصير الملف النشط ثم تُفتح الرئيسية. هذا الملف يشرح أيضًا ورقة اختيار الرمز السري (`pin_setup_sheet.dart`) التي تفتحها الشاشة.

ما أضفناه عن التصميم (لأن Figma لا يحدّد كيف يُدخَل الرمز): ورقة سفلية بخطوتين إدخال ثم تأكيد. وأيقونتا القفل والمجموعة من Material لأنهما لم تُصدَّرا من Figma بعد.

lib/features/onboarding/create_profile_step2_screen.dart — الخطوة 2

`lib/features/onboarding/create_profile_step2_screen.dart`

السلاسة: نقطة اللون `_ColorDot` تنتهي بـ `.pressable()` ويهتزّ الجهاز عند اختيار لون. استيرادا `pressable_scale` و `haptics` في السطرين 25–26.

الأسطر 1–5

```
import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
```

ماذا يفعل؟ استيراد: `foundation` (لـ `kDebugMode`) وMaterial والخدمات (للمرشّحات) وRiverpod وgo_router.

لماذا بهذه الطريقة؟ كل مكتبة لسبب: `kDebugMode` لإظهار سبب الخطأ في التطوير، و `services` لـ `TextInputFormatter` .

الأسطر 7–28

```dart
import '../../app/routes.dart';
import '../../core/db/enums.dart';
import '../../core/strings/ar.dart';
import '../../core/theme/app_colors.dart';
import '../../core/theme/app_text_styles.dart';
import '../../core/theme/profile_palette.dart';
import '../../core/utils/class_code.dart';
import '../../core/widgets/icon_tile.dart';
import '../../core/widgets/mascot.dart';
import '../../core/widgets/app_icon.dart';
import '../../core/widgets/offline_chip.dart';
import '../../core/widgets/rizma_button.dart';
import '../../core/widgets/rizma_card.dart';
import '../../core/widgets/rizma_switch.dart';
import '../../core/widgets/screen_top_bar.dart';
import '../profiles/active_profile_provider.dart';
import '../profiles/pin_setup_sheet.dart';
import '../profiles/profile_providers.dart';
import 'create_profile_draft.dart';
import 'widgets/step_progress_card.dart';
import '../../core/widgets/pressable_scale.dart';
import '../../core/utils/haptics.dart';
```

ماذا يفعل؟ استيراد المسارات والأصول والتعدادات والنصوص والألوان والأنماط ولوحة ألوان الشخصية وأداة رمز الصف والمكوّنات المشتركة ومزوّدات الملف والمسودة.

لماذا بهذه الطريقة؟ كل ما تحتاجه الشاشة من الطبقات الأخرى.

الأسطر 30–37

```dart
/// الشاشة 26 — إنشاء الملف، الخطوة 2 من 2: لون الشخصية + رمز سري + رمز الصف.
class CreateProfileStep2Screen extends ConsumerStatefulWidget {
  const CreateProfileStep2Screen({super.key});

  @override
  ConsumerState<CreateProfileStep2Screen> createState() =>
      _CreateProfileStep2ScreenState();
}
```

ماذا يفعل؟ تعليق وصنف `ConsumerStatefulWidget` (`CreateProfileStep2Screen`) و `createState` .

لماذا بهذه الطريقة؟ حالة محلية (اللون والرمز والنص) مع قراءة مزوّدات.

الأسطر 39–40

```dart
class _CreateProfileStep2ScreenState
    extends ConsumerState<CreateProfileStep2Screen> {
```

ماذا يفعل؟ صنف الحالة.

لماذا بهذه الطريقة؟ يعطي `ref` .

الأسطر 41–45

```
41    final _classCodeController = TextEditingController();
42    ProfileColor _color = ProfileColor.teal;
43    String? _pin;
44    bool _classCodeInvalid = false;
45    bool _submitting = false;
```

ماذا يفعل؟ الحقول: `_classCodeController` ولون الشخصية (افتراضيه teal) و`null` (`_pin` = لا رمز) وعلامتا: رمز الصف غير صالح، وجارٍ الإرسال.

لماذا بهذه الطريقة؟ teal هو لون الهوية الأصلي. `_pin` يحتفظ بالأرقام الأربعة مؤقتًا فقط؛ التجزئة تحدث لاحقًا داخل المستودع.

الأسطر 47–51

```
47    @override
48    void dispose() {
49      _classCodeController.dispose();
50      super.dispose();
51    }
```

ماذا يفعل؟ `dispose` : نتخلّص من الـ controller.

لماذا بهذه الطريقة؟ منع تسريب الذاكرة.

الأسطر 53–57

```
53    Future<void> _onPinToggled(bool enabled) async {
54      if (!enabled) {
55        setState(() => _pin = null);
56        return;
57      }
```

ماذا يفعل؟ `_onPinToggled(false)` : نمسح الرمز ونعود.

لماذا بهذه الطريقة؟ إطفاء المفتاح يلغي الرمز المختار.

الأسطر 58–60

```
58      final pin = await showPinSetupSheet(context);
59      if (!mounted) return;
60      setState(() => _pin = pin);
```

ماذا يفعل؟ تشغيل المفتاح: نفتح ورقة اختيار الرمز وننتظر النتيجة، ثم نحفظها في `_pin` (قد تكون `null` إن أُغلقت).

لماذا بهذه الطريقة؟ إن أغلق الطالب الورقة يبقى المفتاح مطفأً (`mounted`). `_pin == null` يحمي من استعمال الحالة بعد إغلاق الشاشة.

السطر 61

```
61    }
```

ماذا يفعل؟ إغلاق الدالة.

لماذا بهذه الطريقة؟ نهاية `_onPinToggled` .

الأسطر 63–65

```
63    Future<void> _submit() async {
64      final draft = ref.read(createProfileDraftProvider);
65      if (draft == null || _submitting) return;
```

ماذا يفعل؟ `_submit`: نقرأ المسودة؛ إن لم توجد أو كان الإرسال جاريًا نخرج.

لماذا بهذه الطريقة؟ يمنع الضغط المزدوج وإنشاء ملفين.

الأسطر 67–72

```
67      final rawClassCode = _classCodeController.text;
68      if (rawClassCode.trim().isNotEmpty &&
69          !ClassCode.isValid(ClassCode.normalize(rawClassCode))) {
70        setState(() => _classCodeInvalid = true);
71        return;
72      }
```

ماذا يفعل؟ نتحقق من رمز الصف إن كُتب: يُوحَّد ثم يُفحص بـ `ClassCode.isValid`؛ إن كان غير صحيح نعلّم الخطأ ونخرج.

لماذا بهذه الطريقة؟ الحقل الفارغ مقبول (اختياري).

الأسطر 74–77

```
74      setState(() {
75        _submitting = true;
76        _classCodeInvalid = false;
77      });
```

ماذا يفعل؟ نضع علامة الإرسال ونمسح علامة الخطأ.

لماذا بهذه الطريقة؟ يتعطّل الزر أثناء الإنشاء.

السطر 78

```
78      final messenger = ScaffoldMessenger.of(context);
```

ماذا يفعل؟ نحفظ `ScaffoldMessenger` قبل أي `await`.

لماذا بهذه الطريقة؟ لأن `context` قد لا يكون صالحًا بعد الانتظار.

الأسطر 79–88

```
79      try {
80        await ref
81            .read(profileRepositoryProvider)
82            .create(
83              name: draft.name,
84              grade: draft.grade,
85              color: _color,
86              pin: _pin,
87              classCode: rawClassCode,
88            );
```

ماذا يفعل؟ نستدعي `ProfileRepository.create` بالاسم والصف واللون والرمز السري ورمز الصف.

لماذا بهذه الطريقة؟ المستودع يجزّئ الرمز ويوحّد رمز الصف ويولّد رمز الطالب ويجعل الملف نشطًا.

الأسطر 91–89

```
89        ref.read(createProfileDraftProvider.notifier).clear();
90        await ref.read(activeProfileIdProvider.notifier).reload();
91        if (mounted) context.go(AppRoutes.home);
```

ماذا يفعل؟ نمسح المسودة ونحدّث الملف النشط في المزوّد ثم ننتقل للرئيسية إن كانت الشاشة ما زالت قائمة.

لماذا بهذه الطريقة؟ `go` وليس `push` فلا يرجع الطالب لشاشات الإنشاء بزر الرجوع.

الأسطر 93–92

```
92      } catch (error, stackTrace) {
93        debugPrint('فشل إنشاء الملف: $error\n$stackTrace');
```

ماذا يفعل؟ في حال الفشل: نطبع الخطأ وتتبّع المكدّس في السجل.

لماذا بهذه الطريقة؟ لمعرفة السبب الحقيقي (مثل غياب ملفات الويب لـ Drift).

الأسطر 103–94

```
94        messenger.showSnackBar(
95          SnackBar(
96            content: Text(
97              // في وضع التطوير نُظهر سبب الخطأ الحقيقي لتسهيل التشخيص.
98              kDebugMode
99                  ? '${ArStrings.setupCreateFailed}\n$error'
100                 : ArStrings.setupCreateFailed,
101           ),
102         ),
103       );
```

ماذا يفعل؟ نعرض `SnackBar`: في وضع debug النص العام مع سبب الخطأ، وفي release النص العام فقط.

لماذا بهذه الطريقة؟ الطالب لا يرى تفاصيل تقنية، والمطوّر يراها.

الأسطر 106–104

```
104       if (mounted) setState(() => _submitting = false);
105     }
106   }
```

ماذا يفعل؟ نعيد تفعيل الزر ثم نغلق الدالة.

لماذا بهذه الطريقة؟ ليحاول الطالب مرة أخرى.

الأسطر 110–108

```
108   @override
109   Widget build(BuildContext context) {
110     final draft = ref.watch(createProfileDraftProvider);
```

ماذا يفعل؟ `build` ونقرأ المسودة بـ `watch` .

لماذا بهذه الطريقة؟ لنعرض الاسم والصف في المعاينة.

الأسطر 112–115

```
    return Scaffold(
      body: SafeArea(
        child: CustomScrollView(
          slivers: [
```

ماذا يفعل؟ `Scaffold` و `SafeArea` و `CustomScrollView`.

لماذا بهذه الطريقة؟ تمرير يسمح بالتكيّف مع الشاشات القصيرة ولوحة المفاتيح.

الأسطر 116–124

```
            SliverToBoxAdapter(
              child: ScreenTopBar(
                title: ArStrings.createProfileTitle,
                showOffline: false,
                onBack: () => context.canPop()
                    ? context.pop()
                    : context.go(AppRoutes.createStep1),
              ),
            ),
```

ماذا يفعل؟ الشريط العلوي «إنشاء الملف» بلا شارة «دون إنترنت»، وزر الرجوع.

لماذا بهذه الطريقة؟ نفس شريط الخطوة 1 حتى تبدو الخطوتان شاشة واحدة متصلة. الرجوع يعود للخطوة 1 أو ينتقل إليها.

الأسطر 125–126

```
            SliverFillRemaining(
              hasScrollBody: false,
```

ماذا يفعل؟ `SliverFillRemaining(hasScrollBody: false)`.

لماذا بهذه الطريقة؟ يملأ باقي الشاشة فيستطيع `Spacer` دفع الزر للأسفل مع بقاء التمرير ممكنًا على الشاشات القصيرة.

الأسطر 127–149

```
            child: Padding(
              padding: const EdgeInsetsDirectional.fromSTEB(20, 12, 20, 24),
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.stretch,
                spacing: 16,
                children: [
                  const StepProgressCard(
                    label: ArStrings.step2Progress,
                    value: 1,
                  ),
                  Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      Text(
                        ArStrings.setupTitle,
                        style: AppTextStyles.title26,
                      ),
                      Text(
                        ArStrings.setupSub,
                        style: AppTextStyles.caption13,
                      ),
                    ],
                  ),
```

ماذا يفعل؟ حشوة وعمود بمسافة 16: بطاقة التقدّم «الخطوة 2 من 2» ممتلئة، ثم العنوان «اجعل الملف لك» وسطر تحته.

لماذا بهذه الطريقة؟ البطاقة الممتلئة تقول «هذه آخر خطوة».

الأسطر 150–155

```
                  _Preview(
                    color: _color,
                    label: draft == null
                        ? ''
                        : ArStrings.setupPreview(draft.name, draft.grade),
                  ),
```

ماذا يفعل؟ المعاينة: لون الشخصية والنص «الاسم · الصف» (فارغ إن لم توجد مسودة).

لماذا بهذه الطريقة؟ تعرض ما أدخله الطالب فعلًا لا مثال Figma.

الأسطر 156–159

```
                  _ColorPicker(
                    selected: _color,
                    onSelected: (c) => setState(() => _color = c),
                  ),
```

ماذا يفعل؟ منتقي الألوان: الاختيار يغيّر `_color` فتتحدّث المعاينة فورًا.

لماذا بهذه الطريقة؟ `setState` .

السطر 160

```
160                 _PinCard(enabled: _pin != null, onChanged: _onPinToggled),
```

ماذا يفعل؟ بطاقة الرمز السري: المفتاح مشغَّل إن كان `_pin` غير فارغ.

لماذا بهذه الطريقة؟ الحالة مشتقة من `_pin` فلا حالة مكرّرة.

الأسطر 161–169

```
161                 _ClassCodeCard(
162                   controller: _classCodeController,
163                   invalid: _classCodeInvalid,
164                   onChanged: () {
165                     if (_classCodeInvalid) {
166                       setState(() => _classCodeInvalid = false);
167                     }
168                   },
169                 ),
```

ماذا يفعل؟ بطاقة رمز الصف: الـ controller وحالة الخطأ ودالة تمسح الخطأ عند الكتابة.

لماذا بهذه الطريقة؟ حتى لا يبقى الإطار أحمر بعد أن يصحّح الطالب.

الأسطر 170–171

```
170                 const Spacer(),
171                 RizmaButton(
```

ماذا يفعل؟ `Spacer` ثم بداية الزر.

لماذا بهذه الطريقة؟ الزر في أسفل الشاشة كما في التصميم.

الأسطر 172–175

```
172                   label: ArStrings.setupStart,
173                   icon: 'bolt',
174                   onPressed: _submitting ? null : _submit,
175                 ),
```

ماذا يفعل؟ زر «ابدأ التعلّم» ببرق: يتعطّل أثناء الإرسال.

لماذا بهذه الطريقة؟ `null` يعطّل الزر؛ البرق من التصميم الجديد.

الأسطر 176–180

```
176                 Text(
177                   ArStrings.setupChangeLater,
178                   textAlign: TextAlign.center,
179                   style: AppTextStyles.caption13,
180                 ),
```

ماذا يفعل؟ ملاحظة «يمكنك تغيير كل هذا لاحقًا من حسابك.».

لماذا بهذه الطريقة؟ نص تطميني بمحاذاة المنتصف.

الأسطر 181–190

```
                  ],
                ),
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

ماذا يفعل؟ إغلاق الأقواس والدوال والصنف.

لماذا بهذه الطريقة؟ نهاية `CreateProfileStep2Screen`.

الأسطر 192–196

```
class _Preview extends StatelessWidget {
  const _Preview({required this.color, required this.label});

  final ProfileColor color;
  final String label;
```

ماذا يفعل؟ ودجت `_Preview` ولونه ونصه.

لماذا بهذه الطريقة؟ بطاقة المعاينة.

الأسطر 198–209

```
  @override
  Widget build(BuildContext context) {
    return AnimatedContainer(
      duration: const Duration(milliseconds: 150),
      padding: const EdgeInsetsDirectional.fromSTEB(16, 12, 16, 14),
      decoration: BoxDecoration(
        color: color.avatarBackground,
        borderRadius: BorderRadius.circular(26),
        boxShadow: [
          BoxShadow(color: color.fold, offset: const Offset(0, 4)),
        ],
      ),
```

ماذا يفعل؟ `AnimatedContainer` بخلفية لون الطالب الفاتح وزوايا 26 وحشوة، وحافة سفلية بلون الورقة.

لماذا بهذه الطريقة؟ الحركة تجعل تغيّر اللون ناعمًا؛ الحافة الملوّنة تجعل المعاينة ملموسة.

الأسطر 210–223

```
      child: Column(
        spacing: 4,
        children: [
          const Align(
            alignment: AlignmentDirectional.centerStart,
            child: OfflineChip(),
          ),
          Mascot(size: 128, color: color),
          Text(
            label,
            style: AppTextStyles.section18.copyWith(color: color.dark),
          ),
        ],
      ),
```

ماذا يفعل؟ عمود: شارة «دون إنترنت» في الطرف، ثم التميمة 128 بلون الطالب، والاسم بلونه الغامق.

لماذا بهذه الطريقة؟ المعاينة الكبيرة تُري الطالب تميمته قبل إنشاء الملف.

الأسطر 224–226

```
    );
  }
}
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية `_Preview` .

الأسطر 228–232

```
class _ColorPicker extends StatelessWidget {
  const _ColorPicker({required this.selected, required this.onSelected});

  final ProfileColor selected;
  final ValueChanged<ProfileColor> onSelected;
```

ماذا يفعل؟ ودجت `_ColorPicker` وحقلاه.

لماذا بهذه الطريقة؟ اللون المختار ودالة الاختيار.

الأسطر 234–253

```
  @override
  Widget build(BuildContext context) {
    return Column(
      crossAxisAlignment: CrossAxisAlignment.stretch,
      spacing: 10,
      children: [
        Row(
          children: [
            Expanded(
              child: Text(
                ArStrings.setupPickColor,
                style: AppTextStyles.label14.copyWith(color: AppColors.ink),
              ),
            ),
            Text(
              ArStrings.colorName(selected),
              style: AppTextStyles.caption12.copyWith(color: selected.body),
            ),
          ],
        ),
```

ماذا يفعل؟ `build` وصف: «لون التميمة» واسم اللون المختار بلونه.

لماذا بهذه الطريقة؟ اسم اللون مكتوب لمن لا يميّز الألوان.

الأسطر 254–260

```
        // FittedBox: خمس دوائر بعرض 56 لا تتسع في 280، فتتقلّص قليلًا بدل أن تفيض.
        FittedBox(
          fit: BoxFit.scaleDown,
          child: Row(
            mainAxisAlignment: MainAxisAlignment.center,
            spacing: 14,
            children: [
```

ماذا يفعل؟ تعليق ثم `FittedBox(scaleDown)` حول صف الألوان متمركز بمسافة 14.

لماذا بهذه الطريقة؟ خمس دوائر بعرض 56 ومسافات 14 = 336 بكسلًا لا تتسع في 280 على شاشة 320 (فيض 56 بكسلًا اكتشفه `layout_stress_test`). `FittedBox` يصغّرها قليلًا فقط عند الضيق.

الأسطر 261–269

```
              for (final color in ProfileColor.values)
                _ColorDot(
                  color: color,
                  selected: color == selected,
                  onTap: () {
                    Haptics.tap();
                    onSelected(color);
                  },
                ),
```

ماذا يفعل؟ حلقة على `ProfileColor.values` تبني نقطة لكل لون وتعلّم المختار.

لماذا بهذه الطريقة؟ ترتيب التعداد teal أولًا فيظهر الأخضر أقصى اليمين (RTL).

الأسطر 270–276

```
            ],
          ),
        ),
      ],
    );
  }
}
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية `_ColorPicker`.

الأسطر 278–283

```
class _ColorDot extends StatelessWidget {
  const _ColorDot({
    required this.color,
    required this.selected,
    required this.onTap,
  });
```

ماذا يفعل؟ ودجت `_ColorDot` ومعاملاته.

لماذا بهذه الطريقة؟ نقطة لون واحدة.

الأسطر 285–287

```
  final ProfileColor color;
  final bool selected;
  final VoidCallback onTap;
```

ماذا يفعل؟ الحقول.

لماذا بهذه الطريقة؟ اللون والاختيار والإجراء.

الأسطر 289–295

```
  @override
  Widget build(BuildContext context) {
    return Semantics(
      button: true,
      selected: selected,
      label: ArStrings.colorName(color),
      excludeSemantics: true,
```

ماذا يفعل؟ `Semantics` باسم اللون («أخضر»...) وحالة التحديد.

لماذا بهذه الطريقة؟ قارئ الشاشة يقرأ «مرجاني، محدَّد».

الأسطر 296–299

```
      child: GestureDetector(
        onTap: onTap,
        behavior: HitTestBehavior.opaque,
        // دائرة ملموسة بلون التميمة وحافة أغمق، والمختارة بعلامة صح وحلقة.
```

ماذا يفعل؟ `GestureDetector` مع `opaque`.

لماذا بهذه الطريقة؟ كل المساحة قابلة للضغط.

الأسطر 300–304

```
        child: AnimatedContainer(
          duration: const Duration(milliseconds: 150),
          width: 56,
          height: 60,
          alignment: Alignment.topCenter,
```

ماذا يفعل؟ `AnimatedContainer` بعرض 56 وارتفاع 60 ومحاذاة للأعلى.

لماذا بهذه الطريقة؟ الارتفاع الزائد يحجز مكان الحافة السفلية.

الأسطر 305–325

```
          child: Container(
            width: 52,
            height: 52,
            decoration: BoxDecoration(
              color: color.body,
              shape: BoxShape.circle,
              border: selected
                  ? Border.all(color: AppColors.card, width: 3)
                  : null,
              boxShadow: [
                BoxShadow(color: color.dark, offset: const Offset(0, 4)),
                if (selected)
                  BoxShadow(color: color.body, spreadRadius: 3),
              ],
            ),
            alignment: Alignment.center,
            child: selected
                ? const AppIcon('check', size: 24, color: AppColors.card)
                : null,
          ),
        ),
```

ماذا يفعل؟ الدائرة 52 بلون جسم التميمة، بحدّ أبيض 3 للمختارة، وحافة سفلية بلونها الغامق، وعلامة صح للمختارة.

لماذا بهذه الطريقة؟ دوائر ملموسة كما في التصميم الجديد؛ الصح يؤكد الاختيار.

الأسطر 326–329

```
      ),
    ).pressable();
  }
}
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية `_ColorDot`.

الأسطر 331–335

```
class _PinCard extends StatelessWidget {
  const _PinCard({required this.enabled, required this.onChanged});

  final bool enabled;
  final ValueChanged<bool> onChanged;
```

ماذا يفعل؟ ودجت `_PinCard` وحقلاه: مفعّل أم لا ودالة التغيير.

لماذا بهذه الطريقة؟ بطاقة الرمز السري.

الأسطر 337–344

```
  @override
  Widget build(BuildContext context) {
    return RizmaCard(
      padding: const EdgeInsetsDirectional.fromSTEB(16, 14, 16, 14),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        spacing: 10,
        children: [
```

ماذا يفعل؟ `build` و`RizmaCard` بحشوة 16/14 و`Column` بمسافة 10.

لماذا بهذه الطريقة؟ استعملنا البطاقة المشتركة.

الأسطر 345–347

```
          Row(
            spacing: 12,
            children: [
```

ماذا يفعل؟ صف بمسافة 12.

لماذا بهذه الطريقة؟ أول عنصر على اليمين.

الأسطر 348–355

```
            const IconTile(
              iconName: 'lock',
              iconColor: AppColors.purple,
              color: AppColors.purpleTint,
              size: 40,
              iconSize: 20,
              radius: 12,
            ),
```

ماذا يفعل؟ مربّع أيقونة القفل (من مجموعة التصميم) بخلفية بنفسجية فاتحة، 40 وأيقونة 20 وزوايا 12.

لماذا بهذه الطريقة؟ أُضيفت أيقونة `lock` إلى مجموعة التصميم فلم نعد نحتاج أيقونة Material.

الأسطر 356–370

```
            Expanded(
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(
                    ArStrings.setupPinTitle,
                    style: AppTextStyles.body16Strong,
                  ),
                  Text(
                    ArStrings.setupPinSubtitle,
                    style: AppTextStyles.caption13,
                  ),
                ],
              ),
            ),
```

ماذا يفعل؟ `Expanded` فيه عمودان: «رمز سري» و«يمنع الآخرين من فتح ملفك على هذا الجهاز».

لماذا بهذه الطريقة؟ النص يلتفّ حسب العرض المتاح.

الأسطر 371–375

```
            RizmaSwitch(
              value: enabled,
              onChanged: onChanged,
              semanticsLabel: ArStrings.setupPinSwitch,
            ),
```

ماذا يفعل؟ `RizmaSwitch` بالقيمة والدالة ووصف قارئ الشاشة `setupPinSwitch`.

لماذا بهذه الطريقة؟ وصف مختلف عن عنوان البطاقة حتى لا يلتبس على قارئ الشاشة أو الاختبارات.

الأسطر 376–377

```
          ],
        ),
```

ماذا يفعل؟ إغلاق الصف.

لماذا بهذه الطريقة؟ نهاية الصف.

السطر 378

```
          Text(ArStrings.setupPinHint, style: AppTextStyles.caption13),
```

ماذا يفعل؟ الملاحظة «اختياري · يُنصح به للصفوف 7–12».

لماذا بهذه الطريقة؟ تلميح من Figma.

الأسطر 379–383

```
        ],
      ),
    );
  }
}
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية `_PinCard`.

الأسطر 385–390

```
class _ClassCodeCard extends StatelessWidget {
  const _ClassCodeCard({
    required this.controller,
    required this.invalid,
    required this.onChanged,
  });
```

ماذا يفعل؟ ودجت `_ClassCodeCard` ومعاملاته الثلاثة.

لماذا بهذه الطريقة؟ بطاقة رمز الصف.

الأسطر 392–394

```
  final TextEditingController controller;
  final bool invalid;
  final VoidCallback onChanged;
```

ماذا يفعل؟ الحقول.

لماذا بهذه الطريقة؟ الـ controller وحالة الخطأ ودالة التغيير.

الأسطر 396–397

```
  @override
  Widget build(BuildContext context) {
```

ماذا يفعل؟ `build`.

لماذا بهذه الطريقة؟ يعيد البطاقة.

الأسطر 398–402

```
    final borderColor = invalid ? AppColors.coral : AppColors.line;
    final border = OutlineInputBorder(
      borderRadius: BorderRadius.circular(14),
      borderSide: BorderSide(color: borderColor, width: 2),
    );
```

ماذا يفعل؟ نحدّد لون الحد (مرجاني للخطأ وإلا `line`) ونبني الحد المشترك.

لماذا بهذه الطريقة؟ نبنيه مرة ونستعمله في أكثر من حالة.

الأسطر 404–409

```
    return RizmaCard(
      padding: const EdgeInsetsDirectional.fromSTEB(16, 14, 16, 14),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        spacing: 8,
        children: [
```

ماذا يفعل؟ `RizmaCard` و`Column` بمسافة 8.

لماذا بهذه الطريقة؟ نفس هيكل بطاقة الرمز.

الأسطر 410–412

```
          Row(
            spacing: 12,
            children: [
```

ماذا يفعل؟ صف بمسافة 12.

لماذا بهذه الطريقة؟ أيقونة ثم نصوص.

الأسطر 413–420

```
              const IconTile(
                iconName: 'users',
                iconColor: AppColors.teal,
                color: AppColors.tealTint,
                size: 40,
                iconSize: 20,
                radius: 12,
              ),
```

ماذا يفعل؟ مربّع أيقونة المجموعة (من مجموعة التصميم) بخلفية `tealTint`.

لماذا بهذه الطريقة؟ أيقونة `users` صارت في مجموعة التصميم.

الأسطر 421–436

```
            Expanded(
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(
                    ArStrings.setupClassTitle,
                    style: AppTextStyles.body16Strong,
                  ),
                  Text(
                    ArStrings.setupClassSubtitle,
                    style: AppTextStyles.caption13,
                  ),
                ],
              ),
            ),
          ],
```

ماذا يفعل؟ `Expanded` فيه «رمز الصف (اختياري)» و«يعطيك إياه معلمك لتنضم إلى مجموعته».

لماذا بهذه الطريقة؟ تعريف بسيط بالرمز.

السطر 437

```
        ),
```

ماذا يفعل؟ إغلاق الصف.

لماذا بهذه الطريقة؟ نهاية الصف.

الأسطر 438–440

```
        SizedBox(
          height: 50,
          child: TextField(
```

ماذا يفعل؟ `SizedBox` ارتفاعه 50 و`TextField`.

لماذا بهذه الطريقة؟ الارتفاع من Figma.

الأسطر 441–445

```
            controller: controller,
            onChanged: (_) => onChanged(),
            textCapitalization: TextCapitalization.characters,
            textInputAction: TextInputAction.done,
            cursorColor: AppColors.teal,
```

ماذا يفعل؟ الـ controller ودالة التغيير وأحرف كبيرة وإجراء «تم» ولون المؤشر.

لماذا بهذه الطريقة؟ تُستدعى `onChanged` لمسح علامة الخطأ.

الأسطر 446–453

```
            inputFormatters: [
              FilteringTextInputFormatter.allow(RegExp('[A-Za-z0-9-]')),
              LengthLimitingTextInputFormatter(ClassCode.maxLength),
              TextInputFormatter.withFunction(
                (old, value) =>
                    value.copyWith(text: value.text.toUpperCase()),
              ),
            ],
```

ماذا يفعل؟ ثلاثة مرشّحات: السماح بحروف إنجليزية وأرقام وشرطة فقط، وحد أقصى للطول، وتحويل الحروف لكبيرة أثناء الكتابة.

لماذا بهذه الطريقة؟ يمنع الإدخال غير الصالح من الأساس ويطابق صيغة `GZ-5A` .

السطر 454

```
            style: AppTextStyles.body15,
```

ماذا يفعل؟ نمط النص `body15` .

لماذا بهذه الطريقة؟ حجم مناسب.

الأسطر 455–464

```
            decoration: InputDecoration(
              hintText: ArStrings.setupClassHint,
              prefixIcon: const Padding(
                padding: EdgeInsets.all(14),
                child: Text('#', style: TextStyle(color: AppColors.muteText)),
              ),
              hintStyle: AppTextStyles.body15.copyWith(
                fontWeight: FontWeight.w600,
                color: AppColors.placeholder,
              ),
```

ماذا يفعل؟ النص الإرشادي «مثال: GZ-5A» باهتًا، وعلامة «#» في بداية الحقل.

لماذا بهذه الطريقة؟ «#» من التصميم تقول إن المطلوب رمز لا اسم.

الأسطر 465–469

```
              filled: true,
              fillColor: AppColors.ground,
              contentPadding: const EdgeInsetsDirectional.symmetric(
                horizontal: 14,
              ),
```

ماذا يفعل؟ خلفية `ground` وحشوة أفقية 14.

لماذا بهذه الطريقة؟ مظهر Figma.

الأسطر 470–471

```
                border: border,
                enabledBorder: border,
```

ماذا يفعل؟ نفس الحد للحالتين العادية والمفعّلة.

لماذا بهذه الطريقة؟ يلغي حدود Material.

الأسطر 472–478

```
                focusedBorder: border.copyWith(
                  borderSide: BorderSide(
                    color: invalid ? AppColors.coral : AppColors.teal,
                    width: 2,
                  ),
                ),
              ),
```

ماذا يفعل؟ عند التركيز: الحد teal أو مرجاني إن كان الرمز غير صالح.

لماذا بهذه الطريقة؟ تغذية بصرية للتركيز والخطأ.

الأسطر 479–480

```
            ),
          ),
```

ماذا يفعل؟ إغلاق `TextField` و`SizedBox`.

لماذا بهذه الطريقة؟ نهاية الحقل.

الأسطر 481–485

```
          if (invalid)
            Text(
              ArStrings.setupClassInvalid,
              style: AppTextStyles.caption13.copyWith(color: AppColors.coral),
            ),
```

ماذا يفعل؟ إن كان الرمز غير صالح نعرض رسالة الخطأ بلون مرجاني.

لماذا بهذه الطريقة؟ تشرح الصيغة الصحيحة مع مثال.

الأسطر 486–490

```
        ],
      ),
    );
  }
}
```

ماذا يفعل؟ إغلاق الأقواس والصنف.

لماذا بهذه الطريقة؟ نهاية الملف.

lib/features/profiles/pin_setup_sheet.dart — ورقة اختيار الرمز السري

lib/features/profiles/pin_setup_sheet.dart

تُستعمل في إنشاء الملف (الشاشة أعلاه) وفي «نسيت الرمز؟» (شاشة 27). تعيد `String?` : الرمز أو `null` إن أغلقها الطالب.

الأسطر 1–7

```dart
import 'package:flutter/material.dart';

import '../../core/strings/ar.dart';
import '../../core/theme/app_colors.dart';
import '../../core/theme/app_text_styles.dart';
import '../../core/utils/pin_hasher.dart';
import '../../core/widgets/pin_pad.dart';
```

ماذا يفعل؟ استيراد Material والنصوص والألوان والأنماط وأداة الرمز ونقاط/لوحة الرمز.

لماذا بهذه الطريقة؟ تعيد استعمال `PinDots` و `PinPad` نفسهما المستعملين في شاشة 27.

الأسطر 9–10

```dart
/// يفتح ورقة سفلية لاختيار رمز سري جديد (إدخال ثم تأكيد).
/// يعيد الرمز (4 أرقام) أو null إن أغلقها الطالب.
```

ماذا يفعل؟ تعليق الدالة العامة.

لماذا بهذه الطريقة؟ تعيد الرمز المكوّن من 4 أرقام أو `null` .

الأسطر 11–20

```dart
Future<String?> showPinSetupSheet(BuildContext context) {
  return showModalBottomSheet<String>(
    context: context,
    isScrollControlled: true,
    shape: const RoundedRectangleBorder(
      borderRadius: BorderRadius.vertical(top: Radius.circular(28)),
    ),
    builder: (_) => const _PinSetupSheet(),
  );
}
```

ماذا يفعل؟ `showPinSetupSheet` : تفتح `showModalBottomSheet` بخلفية `ground` وزوايا علوية 28 وتبني `_PinSetupSheet` .

لماذا بهذه الطريقة؟ `isScrollControlled` ليأخذ الارتفاع الذي يحتاجه. `Future<String?>` لتنتظرها الشاشة المستدعية.

الأسطر 22–27

```
22  class _PinSetupSheet extends StatefulWidget {
23    const _PinSetupSheet();
24
25    @override
26    State<_PinSetupSheet> createState() => _PinSetupSheetState();
27  }
```

ماذا يفعل؟ ودجت `_PinSetupSheet` بحالة و`createState`.

لماذا بهذه الطريقة؟ نحتاج حالة (الإدخال والتأكيد).

الأسطر 29–33

```
29  class _PinSetupSheetState extends State<_PinSetupSheet> {
30    String _first = '';
31    String _entry = '';
32    bool _confirming = false;
33    bool _mismatch = false;
```

ماذا يفعل؟ الحالة: `_first` الإدخال الأول و`_entry` الجاري وعلامتا: في طور التأكيد، وعدم التطابق.

لماذا بهذه الطريقة؟ الخطوتان تستعملان نفس الشاشة بحالتين.

الأسطر 35–42

```
35    void _onDigit(String digit) {
36      if (_entry.length >= PinHasher.pinLength) return;
37      setState(() {
38        _mismatch = false;
39        _entry += digit;
40      });
41      if (_entry.length == PinHasher.pinLength) _onComplete();
42    }
```

ماذا يفعل؟ `_onDigit`: يتجاهل الزائد، وإلا يضيف الرقم ويمسح علامة الخطأ؛ وعند اكتمال 4 أرقام يستدعي `_onComplete`.

لماذا بهذه الطريقة؟ الإدخال مقيّد بطول الرمز من `PinHasher.pinLength`.

الأسطر 44–47

```
44    void _onDelete() {
45      if (_entry.isEmpty) return;
46      setState(() => _entry = _entry.substring(0, _entry.length - 1));
47    }
```

ماذا يفعل؟ `_onDelete`: يحذف آخر رقم.

لماذا بهذه الطريقة؟ إن كان فارغًا لا يفعل شيئًا.

الأسطر 49–52

```
49    Future<void> _onComplete() async {
50      // وقفة قصيرة ليظهر امتلاء النقطة الرابعة قبل الانتقال.
51      await Future<void>.delayed(const Duration(milliseconds: 150));
52      if (!mounted) return;
```

ماذا يفعل؟ `_onComplete` : ينتظر 150 مللي ثانية ثم يتأكد أن الورقة ما زالت قائمة.

لماذا بهذه الطريقة؟ تظهر النقطة الرابعة ممتلئة قبل الانتقال، و`mounted` يمنع استعمال الحالة بعد الإغلاق.

الأسطر 53–60

```
53      if (!_confirming) {
54        setState(() {
55          _first = _entry;
56          _entry = '';
57          _confirming = true;
58        });
59        return;
60      }
```

ماذا يفعل؟ الإدخال الأول: نحفظه في `_first` ونفرّغ `_entry` وننتقل لخطوة التأكيد.

لماذا بهذه الطريقة؟ خطوة التأكيد تمنع حفظ رمز بخطأ كتابة.

الأسطر 61–64

```
61      if (_entry == _first) {
62        Navigator.of(context).pop(_entry);
63        return;
64      }
```

ماذا يفعل؟ إن طابق التأكيد الإدخال الأول نغلق الورقة ونعيد الرمز.

لماذا بهذه الطريقة؟ `Navigator.pop(value)` يعيد القيمة لمن فتح الورقة.

الأسطر 65–71

```
65      setState(() {
66        _mismatch = true;
67        _entry = '';
68        _first = '';
69        _confirming = false;
70      });
71    }
```

ماذا يفعل؟ وإلا نعلّم عدم التطابق ونمسح كل شيء ونعود لخطوة الإدخال الأولى.

لماذا بهذه الطريقة؟ نبدأ من جديد بدل ترك الطالب عالقًا.

الأسطر 73–75

```
73    @override
74    Widget build(BuildContext context) {
75      return SafeArea(
```

ماذا يفعل؟ `build` و `SafeArea`.

لماذا بهذه الطريقة؟ تحمي من شريط الإيماءات.

الأسطر 76–79

```
76        child: SizedBox(
77          width: double.infinity,
78          child: Padding(
79            padding: const EdgeInsetsDirectional.fromSTEB(24, 24, 24, 16),
```

ماذا يفعل؟ `SizedBox(width: double.infinity)` وحشوة 24/24/24/16.

لماذا بهذه الطريقة؟ العرض الكامل ضروري: بدونه تضيق الورقة لعرض لوحة الأرقام.

الأسطر 80–83

```
80            child: Column(
81              mainAxisSize: MainAxisSize.min,
82              spacing: 14,
83              children: [
```

ماذا يفعل؟ `Column` بحجم أدنى ومسافة 14.

لماذا بهذه الطريقة؟ تأخذ الورقة الارتفاع اللازم فقط.

الأسطر 84–90

```
84                Text(
85                  _confirming
86                      ? ArStrings.pinConfirmTitle
87                      : ArStrings.pinSetupTitle,
88                  textAlign: TextAlign.center,
89                  style: AppTextStyles.section18,
90                ),
```

ماذا يفعل؟ العنوان يتغيّر: «اختر رمزًا سريًّا من 4 أرقام» ثم «أعد إدخال الرمز للتأكيد».

لماذا بهذه الطريقة؟ يعرف الطالب في أي خطوة هو.

الأسطر 91–95

```
91                PinDots(
92                  length: PinHasher.pinLength,
93                  filled: _entry.length,
94                  error: _mismatch,
95                ),
```

ماذا يفعل؟ `PinDots` بعدد الأرقام المُدخلة ولون الخطأ عند عدم التطابق.

لماذا بهذه الطريقة؟ مؤشر بصري للتقدّم.

الأسطر 96–106

```
              SizedBox(
                height: 24,
                child: _mismatch
                    ? Text(
                        ArStrings.pinMismatch,
                        style: AppTextStyles.label14.copyWith(
                          color: AppColors.coral,
                        ),
                      )
                    : null,
              ),
```

ماذا يفعل؟ مساحة ثابتة الارتفاع 24 تعرض «الرمزان غير متطابقين» عند الحاجة.

لماذا بهذه الطريقة؟ مساحة ثابتة حتى لا تقفز اللوحة عند ظهور الرسالة.

السطر 107

```
              PinPad(onDigit: _onDigit, onDelete: _onDelete),
```

ماذا يفعل؟ `PinPad` بدالتي الرقم والحذف.

لماذا بهذه الطريقة؟ نفس لوحة شاشة 27.

الأسطر 108–114

```
            ],
          ),
        ),
      ),
    );
  }
}
```

ماذا يفعل؟ إغلاق الأقواس والصنف.

لماذا بهذه الطريقة؟ نهاية الملف.